Reviewer Pack — Minimal Rank of Twisted Quasi-Symmetric Functions vs ACC⁰ Ci…

Entry 2e6e34058b85 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 11:49:22 UTC

Minimal Rank of Twisted Quasi-Symmetric Functions vs ACC⁰ Circuit Threshold

Entry ID: `2e6e34058b85` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-23 11:49:22 UTC

1. Conjecture

Field A (mathematical branch): Twisted Module Theory **Field B** (complexity object): Complexity Theory: ACC⁰ Circuit Complexity

Statement:

> {'sentence_1': 'The minimal rank of a twisted quasi-symmetric function associated with an explicit Boolean function is linearly proportional to the threshold for ACC⁰ circuits computing that function.', 'sentence_2': 'Specifically, there exists a constant C such that for every explicit Boolean function f computable by an ACC⁰ circuit of depth D, the minimal rank of the corresponding twisted quasi-symmetric function R_f is O(C * log(D)).', 'sentence_3': 'For all instances with n ≤ 40, this proportionality holds within a factor of 2.'}

Rationale (proposer's reasoning):

> {'sentence_1': 'Twisted module theory introduces non-commutative twists that could expose new structural properties in the complexity class ACC⁰.', 'sentence_2': 'Quasi-symmetric functions are known to relate to circuit complexity, and their twisted versions may reveal subtle connections with ACC⁰ depth.', 'sentence_3': 'This conjecture aims to establish a quantitative link between these mathematical objects and the computational complexity of Boolean functions.'}

Taxonomy category: `ACC_SIPSER` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `91a3cf3eb6a499a1`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The minimal rank of a twisted quasi-symmetric function is linearly proportional to the threshold depth of ACC⁰ circuits computing the corresponding Boolean function with a factor of 2 tolerance.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal rank" AND "twisted quasi-symmetric function" AND ACC⁰ - "twisted module theory" AND "ACC⁰ circuit complexity" - "linearly proportional" AND "twisted quasi-symmetric function" AND threshold

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.4s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def compute_acc0_depth(f):
        n = len(f)
        if n == 1:
            return 1
        depth = 1
        while True:
            new_f = []
            for i in range(len(f) // 2):
                new_f.append((f[2*i] + f[2*i+1]) % 2)
            f = new_f
            n //= 2
            if n == 1:
                return depth

    def compute_twisted_quasi_symmetric_rank(f):
        n = len(f)
        rank = 0
        for i in range(2**n):
            if all((f[i] + f[j]) % 2 == (i ^ j) & 1 for j in range(i)):
                rank += 1
        return rank
```

```
def gaussian_elimination(matrix):
    m, n = len(matrix), len(matrix[0])
    for i in range(m):
        max_row = i
        for j in range(i+1, m):
            if abs(matrix[j][i]) > abs(matrix[max_row][i]):
                max_row = j
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
        pivot = matrix[i][i]
        for j in range(n):
            matrix[i][j] /= pivot
        for j in range(m):
            if j != i:
                factor = matrix[j][i]
                for k in range(n):
                    matrix[j][k] -= factor * matrix[i][k]
    return matrix

def rank(matrix):
    m, n = len(matrix), len(matrix[0])
    row_echelon_form = gaussian_elimination(matrix)
    rank = 0
    for i in range(m):
        if any(row_echelon_form[i][j] != 0 for j in range(n)):
            rank += 1
    return rank

n = random.randint(5, 40)
f = generate_boolean_function(n)
depth = compute_acc0_depth(f)
rank_twisted_quasi_symmetric = compute_twisted_quasi_symmetric_rank(f)

if depth == 0:
    return {
        "metric_name": "rank",
        "metric_value": None,
        "instances_tested": 1,
        "conjecture_holds": False,
        "counterexample": "ACC⁰ depth is zero"
    }

C = rank_twisted_quasi_symmetric / math.log(depth)
if C * math.log(depth) * 2 >= rank_twisted_quasi_symmetric:
    conjecture_holds = True
else:
    conjecture_holds = False

return {
    "metric_name": "rank",
    "metric_value": rank_twisted_quasi_symmetric,
    "instances_tested": 1,
    "conjecture_holds": conjecture_holds,
    "counterexample": ""
}
```

```python
if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]]
    if not seeds:
        from sympy import primerange
        seeds = list(primerange(2, 100))[:30]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, **result}}")
        results.append(result)

    if all(r["conjecture_holds"] for r in results):
        mean_value = sum(r["metric_value"] for r in results) / len(results)
        std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(re
        support_fraction = 1.0
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, which means it was unable to complete its execution and provide results. | next: Re-run the test with increased time limits or optimize the code to ensure it completes within a reasonable timeframe.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	18123
2	propose	ollama_remote	glm4:latest	0	0	13521
3	propose	ollama_remote	glm4:latest	0	0	13888

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
4	preregistration	ollama_remote	glm4:latest	0	0	9626
5	novelty	ollama_remote	glm4:latest	0	0	8331
6	novelty	ollama_remote	glm4:latest	0	0	8994
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17547
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11085
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12081
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11118
11	judge	ollama_remote	glm4:latest	0	0	20130

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 144444 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in `research/audit/2e6e34058b85.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/2e6e34058b85.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/2e6e34058b85.tar.gz` (if generated)